

API integration, made understandable.

Turn a button click into useful, trustworthy data — and build interfaces that explain what is happening along the way.

Read the request–
response conversation

Fetch and render useful
JSON data

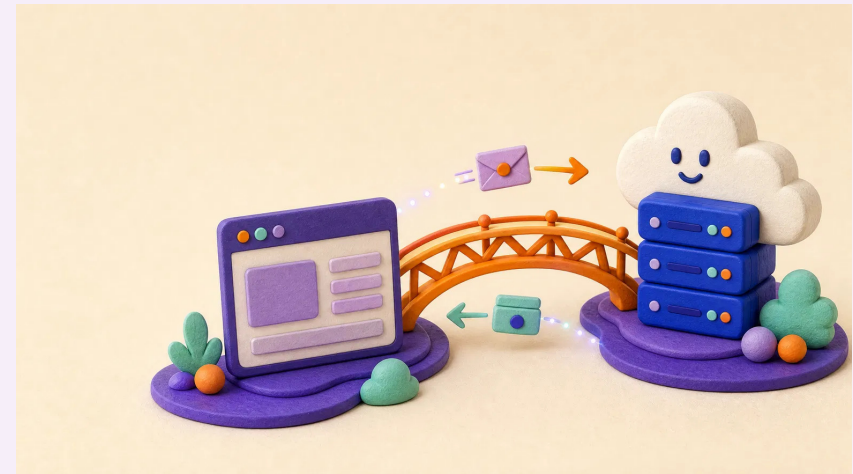
Design for loading,
empty, and error states



From screen to service

An API connects what the user sees to what the server knows.

- 1 The browser sends a **Request** across the network, asking for data or an action.
- 2 The server receives the message, does the heavy lifting, and checks permissions.
- 3 The server sends a **Response** back, delivering the data or confirming the change.



An API is a conversation

The contract defines exactly how to ask and what to expect in return.

```
https://api.campus.edu/v1/events?status=upcoming
```

BASE SERVER

ENDPOINT PATH

PARAMETERS

1. The Address

The **Endpoint** tells the server which specific resource you want to interact with (like events, users, or rooms).

2. The Action

The **Method** (GET, POST, etc.) tells the server what you want to do with that resource.

3. The Promise

The **Response** guarantees the shape of the returned data and a status code explaining what happened.

Choose the right request

Methods declare intent. Status codes reveal the outcome.

The Action (Method)

GET	Read data. "Give me the list of events."
POST	Create data. "Save this new event."
PUT	Update data. "Replace event details."
DELETE	Remove data. "Delete this event."

The Outcome (Status)

200s	Success. The server processed the request.
400	Bad Request. The client sent invalid data.
401 / 403	Unauthorized. Log in or check permissions.
404	Not Found. The resource doesn't exist.
500s	Server Error. The server failed unexpectedly.

JSON: data with shape

JavaScript Object Notation is the universal language of APIs.

- { Objects** hold related details grouped together.
- [Arrays** hold ordered lists of things (like multiple events).
- " Key-Value pairs** map names to strings, numbers, or booleans.



```
{  
  "status": "success",  
  "events": [  
    {  
      "id": 101,  
      "title": "Campus Tech Talk",  
      "isFree": true  
    }  
  ]  
}
```

Fetch your first endpoint

Use JavaScript to ask the server for data and read the response.

```
async function getCampusEvents() {  
  // 1. Ask the server for data  
  const response = await fetch('https://api.c  
  
  // 2. Parse the response into JSON  
  const data = await response.json();  
  
  // 3. Use the data in your app  
  console.log(data.events[0].title);  
}
```

STEP 1: FETCH

Call the **Endpoint URL**. The `await` keyword tells JavaScript to pause and wait for the server to reply.

STEP 2: PARSE

Convert the raw network response into a usable **JSON object**.

STEP 3: USE

Access the data you need using standard object dot-notation.

A reliable interface has states

Because networks are unpredictable, your app must handle the waiting and the failures.



Loading

Give immediate feedback while waiting.



Success

Render the requested JSON data clearly.



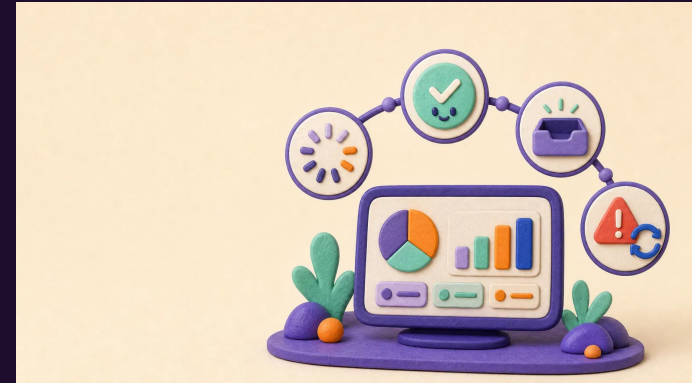
Empty

Explain what to do when there's no data.



Error

Show a friendly message and a way to retry.



Build the state loop

Wrap your fetch in a try/catch block to manage every possible outcome.

```
async function loadData() {
  showLoadingUI();
  try {
    const res = await fetch('/api/data');
    if (!res.ok) throw new Error('Fetch failed');
    const data = await res.json();
    if (data.length === 0) showEmptyUI();
    else showSuccessUI(data);
  } catch (error) {
    showErrorUI(error.message);
  }
}
```



1. Loading First

Show a spinner or skeleton before starting the request.



2. Guard the Try

Check `res.ok` — `fetch()` won't throw for 400/500 responses.



3. Check for Empty

Handle an empty array as a valid (but empty) success case.



4. Catch Everything

The catch block covers network, parsing, and thrown errors.

Protecting the data

APIs aren't always public. Servers need to know exactly who is asking.

API Keys & Tokens

Think of an API key like a digital ID badge. You send it with your request so the server can verify you have permission to access the data.

Headers

Headers are like the envelope of a letter. They carry hidden metadata alongside your request, including your API key and content type.



```
fetch('https://api.campus.edu/secure', {  
  headers: {  
    "Authorization": "Bearer YOUR_KEY_HERE",  
    "Content-Type": "application/json"  
  }  
});
```

Never trust the network

External APIs can change, fail, or return unexpected data. Professional developers build applications that survive bad data.

01

Validate Types

Before running `.map()` on a list of events, ensure the API actually returned an Array. If it returned an Object or Null, your app will crash.

02

Fallback Values

If a user profile API doesn't return an avatar image URL, your UI should gracefully fall back to a default placeholder image instead of showing a broken link.

03

API Versioning

Companies update their APIs. Always connect to a specific version (like `/v1/events`) so future changes to their data shape don't break your live application.

MINI QUIZ

Check your understanding

Discuss these questions with your partner before checking the answers.

QUESTION 01

What does a 404 status code mean when you make a fetch request?

QUESTION 02

What is the standard data format used by modern web APIs?

QUESTION 03

Which JavaScript keyword tells the code to pause and wait for the server to reply?

ANSWER KEY

- 01 Not Found.** The endpoint URL you requested doesn't exist on the server.
- 02 JSON** (JavaScript Object Notation), which uses objects and arrays.
- 03 await.** It is used inside an `async` function to wait for the promise to resolve.

Build a Weather Dashboard

Put your new skills to work by integrating a real public API into a web application.

DELIVERABLES

- An **async function** that fetches data from the provided Weather API endpoint.
- A **Loading state** that displays while the network request is running.
- A **Success state** that parses the JSON and updates the DOM with the temperature and conditions.
- An **Error state** that catches network failures and displays a friendly message.

SUCCESS CRITERIA

- ✓ Uses try/catch blocks correctly.
- ✓ Checks `response.ok` before parsing.

EXTENSION CHALLENGE

Add a text input and a search button. Modify your fetch URL dynamically so users can look up the weather for any city they type in.